TALLER DE PROGRAMACIÓN / CLASE #07

Tuplas, Listas y Diccionarios - Docentes

1. Tuplas: definición y sintaxis
2. Listas: repaso y operaciones más avanzadas
3. Diccionarios: definición y sintaxis
4. Anidamiento de estructuras
5. Recorrido combinado con for
6. Elegir la estructura correcta
7. Ejemplos teórico-prácticos integradores
99. Ejercitación propuesta

1. Tuplas: definición y sintaxis

Una tupla (tuple) es una colección ordenada e inmutable de elementos. A diferencia de las listas, una vez creada, no se puede modificar: no se pueden agregar, eliminar ni reasignar sus elementos.

Sintaxis:

```
coordenadas = (10, 20)
persona = ("Ana", 25, "Rosario")
tupla_un_elemento = (5,)       # la coma es obligatoria, si no, es solo un número entre paréntesis
tupla_sin_parentesis = 1, 2, 3  # los paréntesis son opcionales al crearla
```

Acceso y desempaquetado:

```
punto = (4, 7)
print(punto[0])          # 4 (acceso por índice, igual que en listas)

x, y = punto              # desempaquetado: x = 4, y = 7
print(x, y)
```

¿Por qué usar una tupla en lugar de una lista? Cuando los datos representan algo que no debería cambiar (una coordenada, una fecha, los días de la semana), o cuando se quiere señalar con el propio tipo de dato que esa colección es fija. Además, al ser inmutables, son levemente más eficientes y pueden usarse como claves de diccionario (las listas no).

2. Listas: repaso y operaciones más avanzadas

Como ya vimos, una lista (list) es una colección ordenada y mutable. En este punto ampliamos con herramientas más potentes para trabajar con ellas.

Slicing avanzado:

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numeros[2:7])       # [2, 3, 4, 5, 6]
```

```
print(numeros[:4])         # [0, 1, 2, 3]
print(numeros[6:])         # [6, 7, 8, 9]
print(numeros[::2])        # [0, 2, 4, 6, 8]  (salteando de a 2)
print(numeros[::-1])       # lista invertida completa
```

Comprensión de listas (list comprehension): una forma compacta de crear listas a partir de otra secuencia, aplicando una transformación y/o un filtro en una sola línea.

```
cuadrados = [n ** 2 for n in range(1, 6)]
print(cuadrados)                # [1, 4, 9, 16, 25]

pares = [n for n in range(1, 20) if n % 2 == 0]
print(pares)                     # [2, 4, 6, ..., 18]
```

Nota: La comprensión de listas es equivalente a un for con append(), pero más concisa.

3. Diccionarios: definición y sintaxis

Un diccionario (dict) es una colección de pares clave-valor (desde Python 3.7 mantiene el orden de inserción). Cada clave es única y permite acceder directamente a su valor asociado, sin necesidad de recorrer toda la estructura.

Sintaxis:

```
persona = {
    "nombre": "Carla",
    "edad": 28,
    "ciudad": "Córdoba"
}
```

Acceso, modificación y eliminación:

```
print(persona["nombre"])         # "Carla"

persona["edad"] = 29               # modifica un valor existente
persona["profesion"] = "Ingeniera"  # agrega una clave nueva

del persona["ciudad"]              # elimina una clave
```

Acceso seguro con .get(): evita errores cuando la clave podría no existir.

```
print(persona.get("telefono"))               # None, en vez de lanzar un
error
print(persona.get("telefono",  "Sin  dato"))  #  "Sin  dato"  (valor  por
defecto)
```

Métodos principales:

Método	Descripción
.keys()	Devuelve todas las claves
.values()	Devuelve todos los valores
.items()	Devuelve pares (clave, valor)
.get(clave, default)	Acceso seguro
.pop(clave)	Elimina y devuelve el valor de una clave
.update(otro_dict)	Combina/actualiza con otro diccionario

4**. Anidamiento de estructuras**

Listas, tuplas y diccionarios pueden contenerse entre sí, formando estructuras más complejas que reflejan datos del mundo real (por ejemplo, una lista de personas, cada una representada como un diccionario).

Lista de diccionarios (patrón muy común):

```
alumnos = [
    {"nombre": "Juan", "nota": 8},
    {"nombre": "Lucía", "nota": 6},
    {"nombre": "Martín", "nota": 9},
]

print(alumnos[0]["nombre"])     # "Juan"
```

Diccionario con listas como valores:

```
curso = {
    "materias": ["Matemática", "Programación", "Física"],
     "horarios": [(8, 10), (10, 12), (14, 16)]    # tupla dentro de una lista dentro de un diccionario
}

print(curso["materias"][1])      # "Programación"
print(curso["horarios"][0])       # (8, 10)
```

Idea clave: para acceder a un dato dentro de una estructura anidada, se van "encadenando" los accesos ([índice] o ["clave"]) de afuera hacia adentro.

5. Recorrido combinado con for

Cada estructura tiene su propia forma idiomática de recorrerse con for, y a menudo se combinan entre sí.

```
# Recorrer una lista de tuplas, desempaquetando directamente
puntos = [(1, 2), (3, 4), (5, 6)]
for x, y in puntos:
    print(f"Punto: ({x}, {y})")

# Recorrer un diccionario por claves (por defecto)
persona = {"nombre": "Ana", "edad": 30}
for clave in persona:
    print(clave, "->", persona[clave])

# Recorrer un diccionario por pares clave-valor (forma más clara)
for clave, valor in persona.items():
    print(clave, "->", valor)

# Recorrer una lista de diccionarios
alumnos = [{"nombre": "Juan", "nota": 8}, {"nombre": "Lucía", "nota": 6}]
for alumno in alumnos:
    print(alumno["nombre"], "sacó", alumno["nota"])
```

zip(): combina dos secuencias en pares, muy útil junto con for.

```
nombres = ["Ana", "Luis", "Marta"]
edades = [25, 30, 22]

for nombre, edad in zip(nombres, edades):
    print(f"{nombre} tiene {edad} años")
```

6. Elegir la estructura correcta

Parte de "pensar como programador" es saber qué estructura de datos usar según el problema. Esta tabla resume el criterio:

Estructura	Ordenada	Mutable	Acceso típico	Cuándo usarla
Lista	Sí	Sí	por índice	Colección de elementos que puede crecer/cambiar
Tupla	Sí	No	por índice	Datos fijos, agrupaciones que no deben modificarse
Diccionario	Sí (desde 3.7)	Sí	por clave	Datos identificados por nombre/clave, no por posición

Conversión entre estructuras:

```
lista = [1, 2, 2, 3, 3, 3]
tupla = tuple(lista)               # lista -> tupla
lista_de_nuevo = list(tupla)        # tupla -> lista

claves = ["a", "b", "c"]
valores = [1, 2, 3]
diccionario = dict(zip(claves, valores))    # combinando dos listas en un
diccionario
print(diccionario)   # {'a': 1, 'b': 2, 'c': 3}
```

7. Ejemplos teórico-prácticos integradores

Ejemplo 1 — Tupla para datos fijos, desempaquetada en una función

```
def distancia_al_origen(punto):
    x, y = punto
    return (x ** 2 + y ** 2) ** 0.5

print(distancia_al_origen((3, 4)))   # 5.0
```

Ejemplo 2 — Comprensión de listas dentro de una función

```
def obtener_pares(lista):
    return [numero for numero in lista if numero % 2 == 0]

print(obtener_pares([1, 2, 3, 4, 5, 6, 7, 8]))   # [2, 4, 6, 8]
```

Ejemplo 3 — Diccionario como "registro" con acceso seguro

```
def mostrar_ficha(persona):
    nombre = persona.get("nombre", "Desconocido")
    edad = persona.get("edad", "No informada")
    print(f"{nombre} - {edad} años")

mostrar_ficha({"nombre": "Sofía"})    # Sofía - No informada años
```

Ejemplo 4 (avanzado) — Lista de diccionarios procesada con funciones auxiliares

```
def aprobo(alumno):
    return alumno["nota"] >= 6

def listar_aprobados(alumnos):
    return [alumno["nombre"] for alumno in alumnos if aprobo(alumno)]

curso = [
    {"nombre": "Juan", "nota": 8},
    {"nombre": "Lucía", "nota": 4},
    {"nombre": "Martín", "nota": 9},
```

```
]

print(listar_aprobados(curso))   # ['Juan', 'Martín']
```

Ejemplo 5 (avanzado) — Construcción de un diccionario con funciones de apoyo

```
def calcular_promedio(notas):
    return sum(notas) / len(notas)

def armar_reporte(nombres, notas_por_alumno):
    reporte = {}
    for nombre, notas in zip(nombres, notas_por_alumno):
        reporte[nombre] = calcular_promedio(notas)
    return reporte

nombres = ["Ana", "Luis"]
notas = [[8, 7, 9], [5, 6, 4]]

print(armar_reporte(nombres, notas))
# {'Ana': 8.0, 'Luis': 5.0}
```

99. Ejercitación propuesta

EJ01) Creá una tupla con las coordenadas (3, 8) y desempaquetala en dos variables x e y. Imprimí ambas.

EJ02) Creá un diccionario libro con las claves título, autor y anio, y mostrá cada valor con un print distinto.

EJ03) Dada la lista [5, 12, 8, 21, 3], usá comprensión de listas para crear una nueva lista con el doble de cada valor.

EJ04) Creá un diccionario vacío y, usando un for, agregale como clave cada letra del string "hola" y como valor su posición en la palabra.

EJ05) Dada una lista de tuplas [("Ana", 8), ("Luis", 5), ("Martina", 9)], recorrela con for desempaquetando cada tupla, e imprimí solo los nombres con nota mayor o igual a 7.

EJ06) Definir una función **invertir_diccionario(d)** que reciba un diccionario simple (valores únicos) y devuelva uno nuevo con claves y valores invertidos.

EJ07) Definir una función **contar_ocurrencias(lista)** que reciba una lista de strings y devuelva un diccionario con cuántas veces aparece cada palabra.

EJ08) Dada una lista de diccionarios que representan productos (nombre, precio, stock), escribí una función **valor_total_inventario(productos**) que devuelva la suma de precio * stock de todos los productos.

EJ09) Combinando funciones auxiliares, escribí **agrupar_por_aprobacion(alumnos)** que reciba una lista de diccionarios (nombre, nota) y devuelva un diccionario con dos claves, "aprobados" y "desaprobados", cada una con la lista de nombres correspondiente.

EJ10) Escribí un mini sistema de "carrito de compras": una lista de tuplas (producto, precio, cantidad), y dos funciones — **calcular_subtotal(item)** (recibe una tupla y devuelve precio * cantidad) y **generar_resumen(carrito)** (recorre la lista, arma un diccionario {producto: subtotal} y además devuelve el total general).